PROJECT REPORT

AQI TRENDZ – A SERVERLESS ML PROJECT

SUBMITTED BY: MUHAMMAD YAWAR
DOMAIN: DATA SCIENCE
INSTRUCTOR: MUHAMMAD MUBEEN
COHORT: 10PEARLS SHINE COHORT 08

AQI TRENDZ: Final Project Report

Live App: https:/aqi-trendz-10pearls-cohort8.streamlit.app/

1. Introduction

1.1 Project Overview

AQIPrediction_10Pearls_Cohort8 is a comprehensive, machine-learning-driven web application built to forecast the Air Quality Index (AQI) and five major air pollutants for Karachi, Pakistan. The system predicts air quality for the next 3 Days (72 hours) on an hourly basis. It operates on a fully automated, serverless architecture that fetches live data, engineers complex features, predicts future conditions, and visualizes the results on an interactive dashboard.

1.2 Problem Statement

Karachi is a rapidly expanding metropolis that frequently experiences severe air pollution due to industrial emissions, heavy traffic, and geographical dust. Poor air quality poses severe respiratory and cardiovascular health risks. A lack of hyper-local, easily interpretable, and forward-looking air quality data makes it difficult for citizens and policymakers to take proactive measures.

1.3 Objectives & Our Approach

Our primary objective was to develop a highly accurate, automated, and explainable forecasting system.

Our Approach: We framed this time-series forecasting challenge as a Supervised Multi-Output Regression problem. Rather than predicting just one value, our model is trained to simultaneously output predictions for 5 distinct targets (PM2.5, PM10, NO_2, SO_2, CO) for a given hour. To achieve the 72-hour forecast, we implemented an autoregressive loop, where the model predicts hour 1, feeds that prediction back into its own historical feature set, and then predicts hour 2, repeating this iteratively until hour 72 is reached.

2. Data Methodology

2.1 Raw Data Acquisition

Data is collected programmatically via the Open-Meteo API. We extract two distinct sets of raw data:

- **Pollution Data:** Hourly ground-truth concentrations of PM2.5, PM10, Nitrogen Dioxide (NO_2), Sulphur Dioxide (SO_2), and Carbon Monoxide (CO).
- **Meteorological Data:** Simultaneous readings for Temperature (2m), Relative Humidity (2m), Wind Speed (10m), Surface Pressure, and Precipitation. Weather is a critical driver of pollution dispersion and accumulation.

2.2 Feature Engineering

Raw data alone is insufficient for time-series forecasting because air pollution is highly persistent (pollution lingering from yesterday heavily influences pollution today). We transformed the raw data by engineering specific advanced features:

- **Historical Lags:** `lag_1h`, `lag_2h`, `lag_6h`, and `lag_24h` for every pollutant. This tells the model exactly what the air was like in the recent past.
- **Rolling Statistics:** `rolling_mean_6h`, `rolling_mean_24h`, and `rolling_std_24h`. This smooths out temporary sensor spikes and helps the model recognize the underlying baseline trend and volatility.

2.3 Exploratory Data Analysis (EDA)

Before modeling, an extensive multi-pollutant EDA was conducted to understand the fundamental drivers of pollution in Karachi. Key findings include:

- **The Leading Pollutant:** PM2.5 and PM10 consistently dominate as the primary threats to air quality, repeatedly breaching WHO limits.
- **Pollutant Co-occurrence Signatures:** We identified a strong "traffic signature" (NO_2 vs CO correlating perfectly due to vehicle exhaust) and a "dust/smog signature" (PM2.5 vs PM10).
- **Weekend Shift:** Pollution levels drop noticeably on weekends, proving the massive impact of commuter traffic and industrial activity.
- **Weather Interaction:** Correlation heatmaps confirmed that wind speed generally drives dispersion (lowering AQI), while certain humidity and temperature profiles trap pollutants near the surface.
- **Temporal Heatmaps:** AQI predictably spikes during the morning (8 AM - 10 AM) and evening (5 PM - 8 PM) rush hours.

3. System Architecture & The ML Pipeline

The project implements a modern MLOps architecture using Hopsworks and GitHub Actions. The architecture is divided into three distinct, decoupled ML pipelines:

3.1 The Feature Pipeline (Data Prep & Engineering)

Scheduled to run hourly via GitHub Actions, this pipeline fetches the raw Open-Meteo data, applies the feature engineering logic defined above, and pushes the newly calculated rows to the Hopsworks Feature Store, creating a central repository of clean, model-ready data.

3.2 The Training Pipeline

This pipeline runs periodically on the full historical dataset (~33,000 hourly records). It pulls historical data from the Feature Store, splits it into training and testing sets, trains the models, evaluates them using validation metrics, and pushes the best-performing models to the Hopsworks Model Registry.

3.3 The Batch Inference Pipeline (Prediction)

Running hourly alongside the Feature Pipeline, this script downloads the active production model from Hopsworks. It pulls the latest 24 hours of data from the Feature Store, performs the autoregressive loop to predict the next 72 hours, and caches the results for the Streamlit dashboard to consume.

4. Machine Learning Modeling

4.1 Algorithm Selection & Model Details

Because we need to predict 5 targets simultaneously, we wrapped our base estimators in Scikit-Learn's `MultiOutputRegressor`. We evaluated three different algorithms:

1. XGBoost (Extreme Gradient Boosting): A highly efficient gradient-boosting framework. It builds decision trees sequentially, learning from the errors of previous trees. It is highly effective at capturing non-linear relationships in tabular environmental data.

- **Parameters:** n_estimators=100, learning_rate=0.1, max_depth=6.

2. Random Forest: An ensemble method that builds multiple decision trees in parallel and averages their predictions. It provides highly robust predictions that are resistant to outliers in the sensor data.

- **Parameters:** n_estimators=60, max_depth=15, min_samples_leaf=2.

3. Linear Regression: Used primarily as a baseline model to benchmark the complex ensemble methods against.

4.2 Handling Overfitting & Underfitting

Tuning the models to generalize well without simply memorizing the training data required careful parameter adjustments:

- **Random Forest:** We initially faced underfitting (R^2 dropped to 0.92) because we used max_features=sqrt (which restricts the number of features each tree can see, primarily useful for classification).

Removing this restored accuracy. To prevent overfitting and keep the model size manageable, we restricted the tree depth to 15 and required a minimum of 2 samples per leaf.

- **XGBoost:** We controlled overfitting by limiting the maximum tree depth to 6, preventing it from memorizing noise in the training set.

5. Results & Evaluation

Our models achieved excellent predictive performance on the unseen test data. The system automatically compares models and deploys the one with the highest R^2 score.

5.1 Evaluation Metrics

Based on the final production training run:

- **XGBoost:** Achieved a Mean Absolute Error (MAE) of 10.26 and an R^2 Score of 0.9683.
- **Random Forest:** Achieved a MAE of 10.44 and an R^2 Score of 0.9657.
- **Linear Regression (Baseline):** Achieved an R^2 Score of 0.9628.

5.2 Interpretation of Results

An R^2 score of ~0.96 means that our engineered features successfully explain 96% of the variance in future pollutant levels. The narrow gap between the baseline Linear Regression and the complex tree models suggests that air pollution in Karachi is highly persistent—what the pollution is today is a very strong linear predictor of what it will be tomorrow. However, XGBoost successfully captured the remaining fraction of complex, non-linear weather interactions, making it the superior model.

6. Challenges & Problem Solving

6.1 Hopsworks Integration

- **Schema Mismatches:** Hopsworks is strictly typed. We faced initial failures because the schema expected by the Model Registry did not perfectly match the raw pandas DataFrame. We resolved this by explicitly defining model schemas and creating strict `version=2` feature views.
- **Library Versioning:** We encountered compatibility errors between the `hopsworks` Python client and the backend server, requiring strict dependency pinning in our `requirements.txt`.

6.2 Model File Sizes

Random Forest models are notoriously large. Our initial unrestricted Random Forest model exceeded Hopsworks upload limits and GitHub Actions memory constraints.

Solution: We aggressively compressed the model file using `joblib` (compress=3) and capped the forest at 60 trees. This reduced the file size from over 1GB to a highly manageable ~25MB without sacrificing our 0.96 R^2 score.

7. User Interface & Explainability

7.1 Interactive Streamlit Dashboard

The end product is a sleek, dark-themed Streamlit dashboard. It features interactive Plotly charts that allow users to hover over exact hours to see expected pollutant levels. The charts automatically draw US EPA Zone Bands (Good, Moderate, Unhealthy, Hazardous) based on the data range, providing immediate visual context for the severity of the pollution.

7.2 Explainable AI (SHAP)

Machine Learning is often criticized as a black box. To build user trust, we integrated SHAP (SHapley Additive exPlanations).

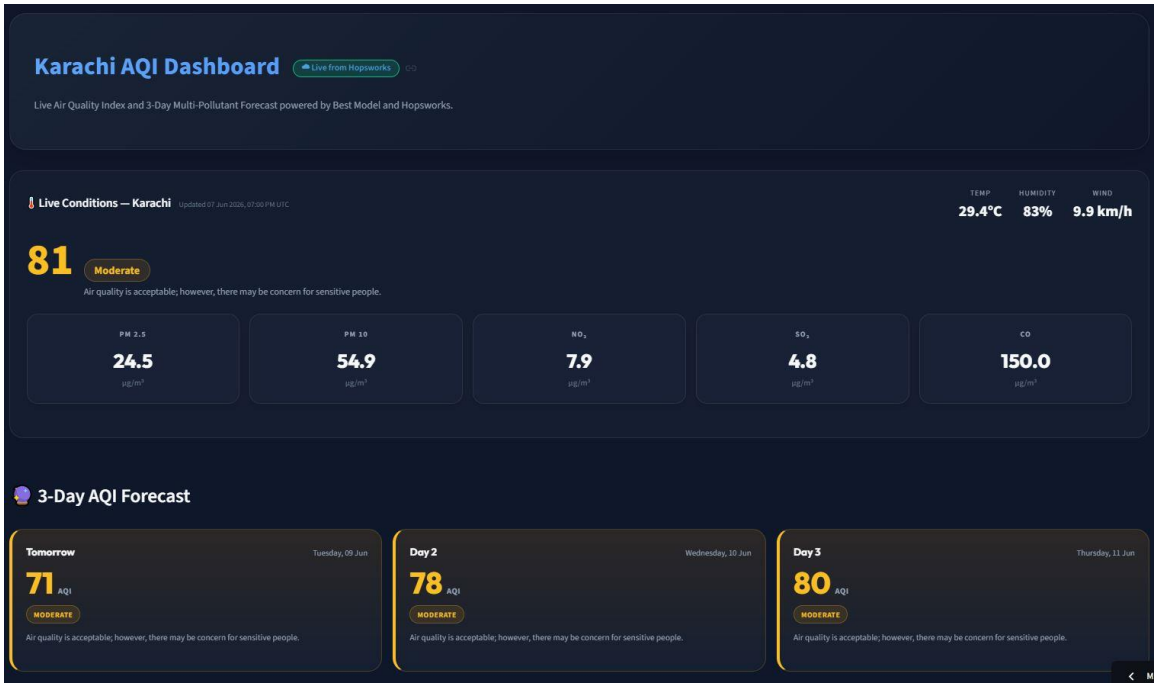
A dedicated page on the dashboard breaks down the model's logic, visually explaining how it arrived at a prediction. For example, the SHAP beeswarm plot explicitly shows users that high wind speeds push pollution predictions down (dispersion), while high historical lags push pollution predictions up (persistence).

8. Live Dashboard Output

The following has the live dashboard output for the AQI TRENDZ for Karachi. The screenshots are taken for 7 June 2026

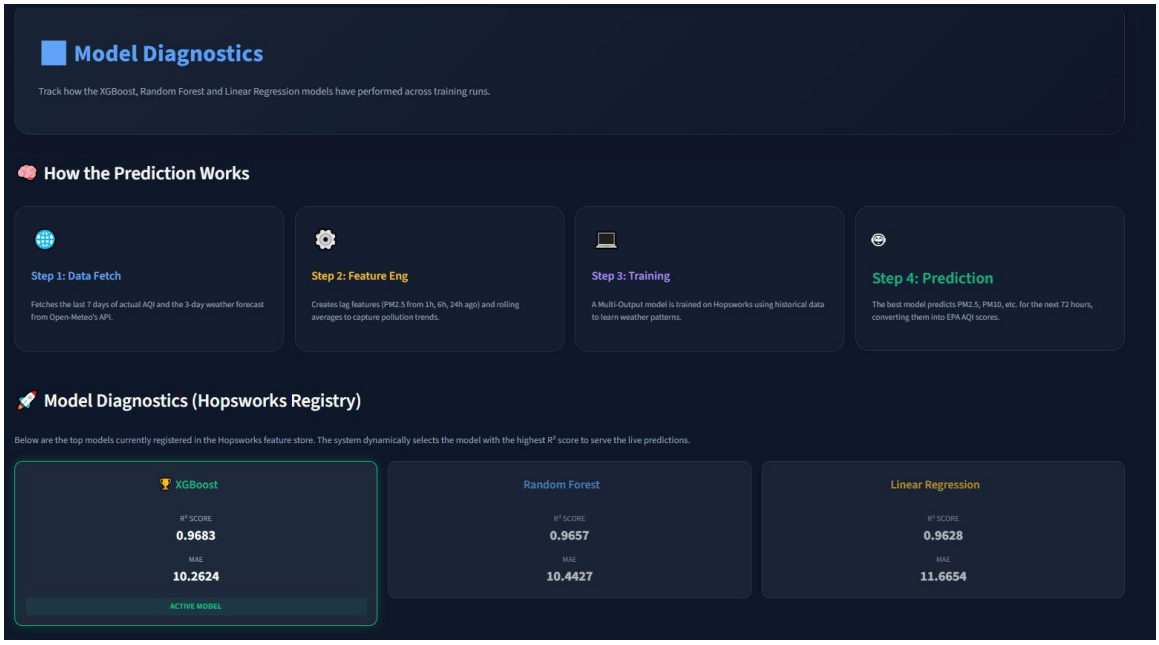
8.1 Dashboard Forecast Screenshot

The screenshot shows live Karachi AQI with with the future 3 day predictions.



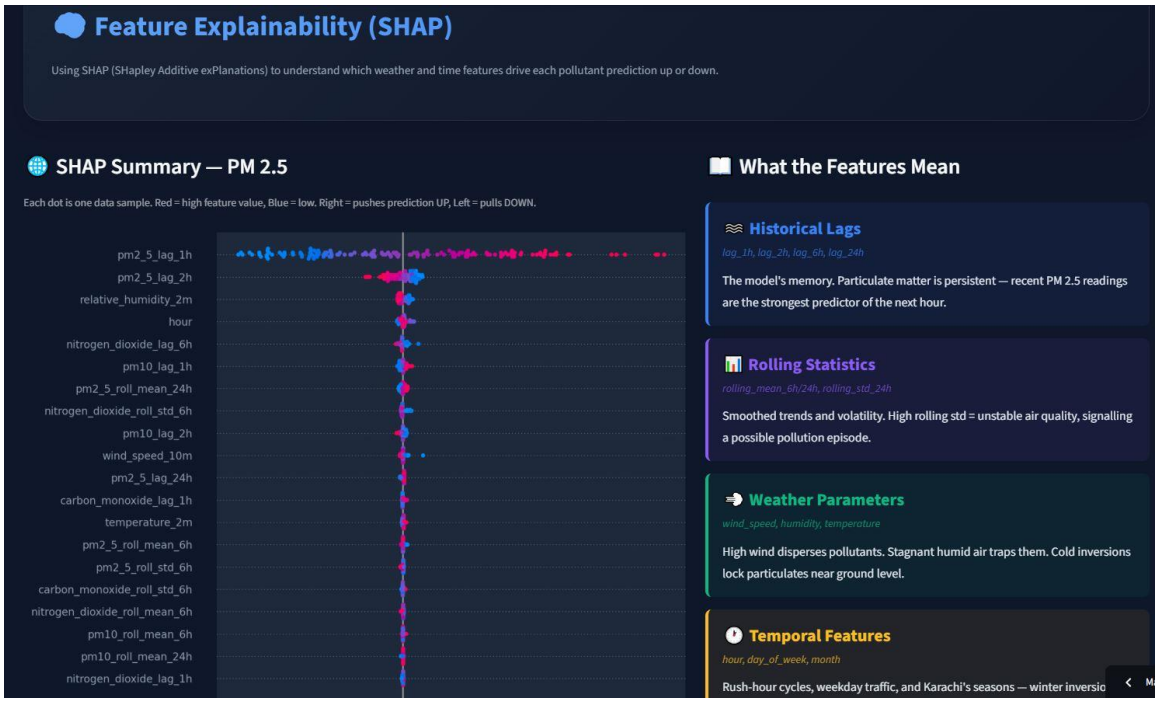
8.2 Model Diagnostics Screenshot

The screenshot shows the latest model results and the model selected for the AQI prediction



8.3 SHAP Analysis Screenshot

The Screenshot shows the SHAP analysis of the PM2.5 as it's the feature with most overwhelming impact on the AQI results



9. Conclusion

The project successfully bridges the gap between complex data science and actionable public information. By fully automating the three distinct ML pipelines via GitHub Actions and Hopsworks, the system requires zero manual maintenance. The integration of advanced feature engineering, robust XGBoost modeling, and SHAP explainability ensures that the citizens of Karachi have access to accurate, transparent, and potentially life-saving air quality forecasts.